

Aegis-Swarm DAH 2026 예선 제출 보고서 초안

DAH 2026 예선 제출 보고서 초안 | Safe Local Red-Blue Self-Play MVP

확인 기준: DAH 2026 공식 홈페이지 <https://dah.ai.kr/guide>, <https://dah.ai.kr/schedule>

확인일: 2026-05-20

1. 프로젝트 개요

Aegis-Swarm은 DAH 2026의 "AI 공격 에이전트 vs AI 방어 에이전트" 구도에 맞춘 안전한 Red-Blue Self-Play 프레임워크다. 방산 임무 환경에서 보안 대응은 탐지 정확도뿐 아니라 임무 지속성과 SLA를 함께 고려해야 한다. 본 프로젝트는 UAV/UGV 임무 서비스를 로컬로 시뮬레이션하고, Red Agent가 안전한 이상 이벤트를 생성하며, Blue Agent가 SLA-aware policy로 방어 액션을 선택하는 구조를 제공한다.

본 MVP는 실제 공격 도구가 아니다. 모든 Red behavior는 SQLite에 기록되는 로컬 시뮬레이션 이벤트이며 외부 시스템 공격, 스캔, 취약점 악용, 자격증명 공격, 악성코드 행위를 수행하지 않는다.

2. 문제 정의

방산 임무 서비스에서는 "무조건 차단"이 항상 좋은 방어가 아니다. 강한 차단은 임무 지연, telemetry 단절, 서비스 재시작으로 이어져 SLA를 떨어뜨릴 수 있다. 반대로 SLA만 보호하면 공격 또는 이상 징후에 대한 대응이 늦어진다.

Aegis-Swarm은 다음 질문에 답하는 것을 목표로 한다.

- Red anomaly가 발생했을 때 Blue Agent가 어떤 신호를 근거로 위험도를 계산하는가?
- 방어 액션이 SLA와 임무 지속성에 어떤 영향을 주는가?
- Commander Agent가 보안 강화와 복구 우선순위를 어떻게 조정하는가?
- 라운드별 scoring을 통해 방어 의사결정을 어떻게 설명 가능하게 만들 수 있는가?

3. 공식 제출 요구사항 대응

공식 안내에서 확인한 예선 준비 관점은 다음과 같다.

- 팀 구성: 팀당 최대 4인
- 접수 마감: 2026-06-14 23:59
- 예선 보고서 제출 기간: 2026-06-15부터 2026-07-10 23:59까지
- 예선 보고서 형식: 통합 PDF 1부, 최대 50MB

- 부가자료: 운영팀이 접근 가능한 링크 제출 가능
- 학생 증빙: 재학증명서 또는 교수추천서, 전원 통합 PDF 1부, 최대 50MB
- 본선 진출팀 발표: 2026-07-31
- 본선 및 시상식: 2026-08-21

Aegis-Swarm 제출 패키지는 위 기준에 맞춰 다음 산출물로 정리한다.

- 예선 보고서 PDF: 본 문서를 기반으로 작성
- 코드 저장소 또는 압축 파일: FastAPI, Streamlit, SQLite, pytest, Docker Compose 포함
- 부가자료 링크: 데모 영상, 스크린샷, 실행 로그, GitHub 또는 공유 드라이브
- 증빙 PDF: 팀원 자격에 따라 별도 준비

4. 아키텍처

```
FastAPI Mission Service
├── Red Agent
├── Blue Agent
├── Commander Agent
├── SQLite event/action/score tables
├── Competition Adapter
│   ├── LocalSimulatorAdapter
│   └── CompetitionStubAdapter
└── Streamlit DAH Scoring Dashboard
```

Mission Service

FastAPI 기반 로컬 API 서버다. `/health`, `/mission/status`, `/vehicle/state`, `/telemetry`, `/simulate/event`, `/selfplay/round`를 제공한다. 모든 명령은 시뮬레이션 상태 전이만 수행하며 시스템 명령을 실행하지 않는다.

Red Agent

Red Agent는 안전한 이벤트만 생성한다. 지원 이벤트는 `TRAFFIC_SPIKE`, `AUTH_ANOMALY`, `TELEMETRY_INCONSISTENCY`, `SERVICE_DEGRADATION`, `MISSION_COMMAND_ANOMALY`, `LOG_NOISE`다. Commander mode와 현재 SLA에 따라 이벤트 강도를 조정한다.

Blue Agent

Blue Agent는 최근 이벤트를 읽고 다음 component score를 계산한다.

- `auth_failure_score`
- `traffic_spike_score`
- `latency_score`

- telemetry_inconsistency_score
- error_rate_score

이후 risk score를 계산하고 SLA-aware policy에 따라 OBSERVE_ONLY, APPLY_RATE_LIMIT, ISOLATE_TELEMETRY_STREAM, RESTART_SERVICE, ROLLBACK_VERSION 등 방어 액션을 선택한다.

Commander Agent

Commander Agent는 SLA와 최근 점수를 바탕으로 BALANCED, RECOVERY_FIRST, DEFENSE_HARDENING, RED_EXPLORATION 중 하나를 선택한다. 이를 통해 self-play가 단순 랜덤 이벤트가 아니라 점수와 임무 상태를 반영하는 전략 루프로 보이게 한다.

Competition Adapter

본선 환경이 공개되면 CompetitionStubAdapter를 실제 대회 API adapter로 교체한다. 현재는 외부 호출을 하지 않는 stub이며, 로컬 실행 기본값은 LocalSimulatorAdapter다.

5. DAH 평가축 대응

Aegis-Swarm은 DAH식 평가 관점을 다음처럼 모델링한다.

- Attack Score: Red anomaly severity와 시나리오 강도
- Defense Score: Blue action이 이벤트 유형에 적절한지
- SLA Score: status code, latency, mission status, sla_ok 기반 가용성
- Recovery Score: 이전 SLA 대비 회복성
- False Positive Penalty: 정상 또는 노이즈 이벤트에 과잉 대응했을 때의 비용
- Total Utility: 방어, 공격, SLA, 복구, false positive를 결합한 통합 점수

/selfplay/round 응답에는 dah_scores와 strategy_notes가 포함되어 심사자가 각 라운드의 판단 근거를 확인할 수 있다.

6. 구체적 시뮬레이션 설계

Aegis-Swarm의 시뮬레이션은 단순 로그 생성이 아니라 "임무 서비스 상태 - Red anomaly - Blue 방어 판단 - Commander 운영 모드 - 점수 변화"가 한 라운드 안에서 연결되는 구조다. 심사자는 dashboard 또는 /selfplay/round API를 통해 한 번의 라운드가 어떤 근거로 진행됐는지 확인할 수 있다.

라운드 진행 흐름

- Mission Service가 현재 UGV 임무 상태를 제공한다. 상태에는 mission status, position, battery, communication status, SLA 상태가 포함된다.
- Commander Agent가 최근 SLA score, defense score, red success rate를 보고 운영 모드를 선택한다.
- Red Agent는 Commander mode에 따라 안전한 로컬 anomaly event를 생성한다.
- Event Generator는 event type, severity, latency, status code, mission status, sla_ok를 포함한 이벤트를 SQLite에 기록한다.
- Blue Agent는 최근 이벤트를 읽고 component score를 계산한다.
- Blue Agent는 risk score와 SLA를 동시에 고려해 방어 액션을 선택한다.
- Action Registry는 실제 시스템 명령을 실행하지 않고 로컬 mission state만 변경한다.
- Scoring 모듈이 attack, defense, SLA, recovery, false positive penalty, total utility를 계산한다.

시뮬레이션 입력 신호

Blue Agent가 보는 입력은 다음처럼 방산 임무 서비스에서 관찰 가능한 운영 신호로 제한된다.

- latency_ms: 임무 API 응답 지연
- status_code: 서비스 오류율 추정
- mission_status: ACTIVE, DEGRADED 등 임무 지속 상태
- sla_ok: 현재 라운드의 SLA 만족 여부
- event_type: 안전한 anomaly category
- severity: Red Agent가 생성한 시뮬레이션 강도
- recent score history: 최근 defense score와 SLA score

상태 전이 예시

Traffic spike 라운드에서는 latency가 상승하지만 mission_status가 ACTIVE로 유지된다. Blue Agent가 APPLY_RATE_LIMIT를 선택하면 Action Registry는 "향후 traffic impact 감소"를 의미하는 로컬 상태만 기록한다. 실제 네트워크 rate limit 명령은 실행하지 않는다.

Telemetry inconsistency 라운드에서는 mission_status는 ACTIVE지만 telemetry 신뢰도가 낮아진 상황을 표현한다. Blue Agent가 ISOLATE_TELEMETRY_STREAM을 선택하면 anomaly가 containment된 것으로 기록하고, scoring은 telemetry event에 맞는 대응을 defense score에 반영한다.

Service degradation 라운드에서는 latency와 status code가 동시에 악화되어 SLA score가 낮아질 수 있다. 이때 Commander Agent는 RECOVERY_FIRST로 전환하고, Blue Agent는 차단형 액션보다 RESTART_SERVICE 또는 ROLLBACK_VERSION 같은 복구 중심 액션을 우선한다. 이 전이 덕분에 "보안 대응이 임무 가용성을 망치지 않는가"를 데모에서 확인할 수 있다.

7. AI 에이전트의 장점 및 차별점

Aegis-Swarm의 핵심 장점은 공격 탐지 자체보다 "임무 지속성을 고려한 방어 의사결정"에 있다. 예선 MVP는 복잡한 black-box ML 대신 설명 가능한 rule-based self-play로 구성했지만, 구조적으로는 본선 환경의 로그와 score API를 연결할 수 있게 설계되어 있다.

SLA-aware Blue Agent

Blue Agent는 위험도가 높다고 항상 가장 강한 대응을 선택하지 않는다. 동일한 anomaly라도 SLA가 충분히 안정적이면 containment나 rate limit을 선택하고, SLA가 이미 나쁘면 blocking보다 recovery를 우선한다. 이 방식은 방산 임무 환경에서 중요한 mission availability를 직접 반영한다.

Explainable Risk Decomposition

위험도는 하나의 숫자로만 출력되지 않고 auth_failure_score, traffic_spike_score, latency_score, telemetry_inconsistency_score, error_rate_score로 분해된다. 따라서 심사자는 Blue Agent가 "왜 이 액션을 선택했는지"를 로그와 dashboard에서 추적할 수 있다.

Commander-driven Self-Play

Commander Agent는 Red와 Blue가 무작위로 움직이지 않도록 운영 방향을 조정한다. SLA가 나쁘면 RECOVERY_FIRST, Blue 실패가 누적되면 DEFENSE_HARDENING, Red 성공률이 낮으면 RED_EXPLORATION으로 전환한다. 이 구조는 self-play가 단순 랜덤 이벤트 생성이 아니라 점수 기반 curriculum처럼 동작하게 만든다.

Safe-by-design Red Agent

Red Agent는 실제 공격을 수행하지 않고 TRAFFIC_SPIKE, AUTH_ANOMALY, TELEMETRY_INCONSISTENCY 같은 안전한 category event만 생성한다. 따라서 대회 예선 단계에서 아이디어를 명확히 보여주면서도 외부 시스템에 위해를 주지 않는다.

Competition Adapter 확장성

에이전트의 판단 로직은 BaseCompetitionAdapter 인터페이스 뒤에 분리되어 있다. 현재는 LocalSimulatorAdapter가 SQLite 기반 로컬 환경을 감싸고, 본선 API가 공개되면 CompetitionStubAdapter 자리에 실제 adapter를 연결하면 된다. 즉, MVP는 "로컬 데모"에 머무르지 않고 대회 runtime으로 옮겨갈 수 있는 구조를 미리 갖춘다.

제출 데모에서 강조할 포인트

- Red event가 안전한 로컬 anomaly인지 확인할 수 있다.
- Blue action이 event type과 SLA 조건에 따라 달라진다.
- Commander mode가 score history에 따라 바뀐다.

- Dashboard에서 Attack, Defense, SLA, Total Utility가 동시에 보인다.
- /selfplay/round 응답에 strategy note가 포함되어 판단 근거가 설명된다.

8. 데모 시나리오

Scenario A: Traffic Spike

Red Agent가 TRAFFIC_SPIKE 이벤트를 생성한다. Blue Agent는 traffic_spike_score와 latency_score를 보고 SLA가 안정적이면 APPLY_RATE_LIMIT를 선택한다. 목표는 방어 대응을 하면서도 mission status를 ACTIVE로 유지하는 것이다.

Scenario B: Telemetry Inconsistency

Red Agent가 TELEMETRY_INCONSISTENCY 이벤트를 생성한다. Blue Agent는 telemetry_inconsistency_score가 높을 때 ISOLATE_TELEMETRY_STREAM을 선택한다. 목표는 telemetry 무결성 이상을 containment하면서 임무 중단을 피하는 것이다.

Scenario C: Service Degradation

Red Agent가 SERVICE_DEGRADATION 이벤트를 생성한다. SLA가 낮아지면 Commander는 RECOVERY_FIRST로 전환하고, Blue Agent는 차단보다 RESTART_SERVICE 또는 ROLLBACK_VERSION 같은 복구 중심 액션을 우선한다.

9. 안전성 선언

본 프로젝트는 다음 기능을 포함하지 않는다.

- 실제 해킹 또는 취약점 악용
- 포트 스캔, 네트워크 스캔
- 브루트포스, credential attack, credential theft
- malware, persistence, lateral movement
- 외부 시스템 공격 또는 침투 행위

모든 Red behavior는 로컬 DB에 기록되는 시뮬레이션 이벤트다. Competition Adapter의 stub도 외부 API를 호출하지 않는다.

10. 실행 및 검증

로컬 실행:

```
pip install -r requirements.txt
uvicorn mission_service.app:app --reload --port 8000
streamlit run dashboard/streamlit_app.py
```

Docker 실행:

```
docker compose up --build
```

검증 명령:

```
pytest
curl http://localhost:8000/health
curl -X POST http://localhost:8000/selfplay/round
```

검증 완료 상태:

- clean clone 검증 통과
- fresh venv dependency install 통과
- pytest 11개 통과
- Docker Compose build/run 통과
- FastAPI 8000 확인
- Streamlit 8501 확인

11. 본선 확장 계획

본선 환경이 공개되면 다음 항목만 교체한다.

- 이벤트 수집: Competition API에서 실시간 로그 수신
- 미션 상태 조회: 대회 runtime 상태 조회
- 방어 액션 제출: Blue action을 대회 API로 제출
- 점수 조회: 공식 scoring API 반영

에이전트 판단 루프, SLA-aware policy, scoring dashboard는 유지한다. 이 구조는 예선 로컬 MVP와 본선 runtime을 같은 인터페이스로 연결하기 위한 준비다.